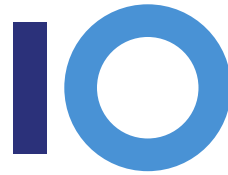




Универзитет „Св. Кирил и Методиј“ во Скопје
**ФАКУЛТЕТ ЗА ИНФОРМАТИЧКИ НАУКИ И
КОМПЈУТЕРСКО ИНЖЕНЕРСТВО**



ПРИСТАПИ ЗА СИМУЛАЦИЈА НА КОЛИЗИЈА НА МЕКИ ТЕЛА

—— дипломска работа ——

Ментор
проф. д-р Бобан Јоксимоски

Кандидат
Давид Стојкоски
206029

мај 2026

Апстракт

Прецизната тридимензионална реконструкција е еден од клучните и најистрајните предизвици во полето на компјутерската визија и компјутерската графика. Целта на реконструкцијата е креирање на веродостојна дигитална репрезентација на објекти или сцени од реалниот свет, користејќи податоци добиени од дводимензионални медиуми, како фотографии или видео-записи. Процесот се базира на инверзија на начинот на кој функционира перцепцијата, односно трансформација на дводимензионални податоци во тридимензионални структури разбирливи за компјутерските системи.

Во рамките на оваа дипломска работа се анализираат трите водечки пристапи на реконструкција: класичната техника на фотограметрија, како и современите пристапи базирани на машинско учење: Neural Radiance Fields (NeRF) и Gaussian splatting. Ќе се испитуваат нивните методолошки пристапи, потреби од податоци, пресметковна ефикасност и визуелен квалитет на конечните резултати, со цел да се добие подлабоко разбирање на предностите и недостатоците на поединечните методи за најпрецизно да се одлучи кој е најсоодветен метод за користење во различни сценарија. Посебен акцент се става на нивната примена во здравството, археологијата, дигитализацијата на културното наследство, виртуелната и проширената реалност, како и потенцијалната комерцијална употреба.

Клучни зборови: *Фотограметрија, Neural radiance fields, Gaussian splatting, тридимензионална реконструкција, машинско учење, дигитализација*

Содржина

1	Вовед	1
1.1	Мотивација	1
1.2	Проблемот на колизии за деформабилни тела	2
1.3	Цели и преглед на трудот	3
2	Позадина и поврзани дела	5
2.1	Парадигми за симулација на меки тела во реално време	5
2.1.1	Системи на маса и пружина	5
2.1.2	Метод на конечни елементи (Finite Element Method)	6
2.1.3	Спојување на облици (Shape matching)	6
2.1.4	Динамика базирана на позиции (Position-Based Dynamics)	6
2.1.5	Проширена диманика базирана на позиции (Extended PBD)	7
2.2	Историја на методи базирани на позиција	7
2.3	Техники на колизии во физички симулации	8
2.3.1	Тврди наспроти меки тела	8
2.3.2	Дискретна наспроти континуирана детекција	9
2.3.3	Парадигми за реакција на колизија	9
2.3.4	Пристапи за само-судир	10
3	Математички и физички основи	11
3.1	Њутново движење и временско интегрирање	11
3.1.1	Експлицитен (forward) Ојлер	11
3.1.2	Симплектички(полуимплицитен) Ојлеров метод	12
3.1.3	Верлеова интеграција	12
3.2	Динамика заснована на ограничувања (Constraint-based dynamics)	13
3.2.1	Ограничувања	13
3.2.2	Лагранжови множители	13
3.2.3	Гаус-Сајделова итерација	13
3.3	XPBD формулацијата	14
3.3.1	Усогласеност (compliance)	14
3.3.2	Терминот alpha	14
3.3.3	Акумулиран Лагранж мултипликатор	15

3.3.4	Ажурирањето по итерација	15
3.3.5	Ажурирање на брзината	15
3.3.6	Зошто функционира цврстината независна од итерацијата	15
3.3.7	Потчекори наспроти итерации	16
4	Модел на меко тело	17
4.0.1	Зошто празна обвивка не може да симулира меко тело	17
4.0.2	Тетраедрализација (tetrahedralization)	17
4.0.3	Делови на мекото тело	18
4.1	Репрезентација на честички	18
4.2	Тетраедрализација	19
4.2.1	TetGen	19
4.2.2	Ограничувања	20
4.3	Ограничувања на рабови	20
4.3.1	Дедупликација	20
4.3.2	Ограничување и градиент	20
4.3.3	Чекор во решавач	21
4.3.4	Ограничување на површински рабови	21
4.3.5	Однесување и нагодување	22
4.4	Волуменски ограничувања	22
4.4.1	Скаларен троен производ	22
4.4.2	Градиенти	22
4.4.3	Чекор во решавачот	23
4.4.4	Инверзија и дегенерација	23
4.4.5	Сооднос од 0.1 за усогласување	23
4.5	Циклусот на XPBD решавачот	23
5	Техники за детекција на колизии	25
5.1	Забрзување во широка фаза (broad-phase acceleration)	25
5.1.1	BVH за статични мрежи	26
5.1.2	Просторен хаш за површината на деформираното меко тело	27
5.2	Тесна фаза: аналитички примитиви	28
5.2.1	SDF апстракција	28
5.2.2	SDF сфера	29
5.2.3	Тесна фаза: колизија на триаголник и мрежа	29
5.2.4	Најблиска точка на триаголник (Воронои метод)	29
5.2.5	Проблем на дисконтинуитет на знак	30
5.2.6	Псевдо-нормали	30
5.2.7	Резултат	31
6	Техники за одговор на колизија	32
6.0.1	Методи со пенали	32
6.0.2	Импулсни методи	33
6.0.3	Контакт базиран на ограничувања	33
7	Имплементација и рендерирање	34

7.1	Технологии	34
7.2	Структура на модули	34

1. Вовед

1.1 Мотивација

Меките, деформабилни тела се голем дел од секојдневниот живот: ткаенина, мускули, кожа, перници, душеци, гуми, кал. Сепак, повеќе децении во компјутерската графика во реално време (real-time CG) приказот овие тела биле третирано како графички трикови наместо физички објекти. За претставување на ткаенина во компјутерски анимации се правеле трикови со vertex shaders и секоја деформација била мануелно креирана од аниматор кој одел слика по слика за да добие прифатлив резултат. Преминувањето кон физичкото симулирање на меки тела е постепена, но сега се користи широко.

Првите техники за симулации на меки тела во реално време во изработка на анимација се на Прово [1], потоа следи подобрување со имплицитното интегрирање на ткаенина на Бараф и Виткин [2] и position-based dynamics на Милер [3]. Секој од овие трудови додаваат нови можности за интерактивни деформации. Моменталната генерација, изградена врз XPBD [4], конечно дозволува реалистични симулации со интерактивна брзина.

Во видео игрите е зголемена побарувачката за интерактивност со светот. Модерните играчки погони?? како Unreal Engine 5, Unity и Frostbite доаѓаат со имплементирани симулации на ткаенина; игрите почесто вклучуваат интеракции со меки тела (деформабилни терени, органски околии, оштетување на каросерија во тркачки игри). Алатки како NVIDIA PhysX, модулот за меки тела на Bullet и Havok Cloth ги направија меките тела честа но се уште скапа одлика.

Во виртуелната реалност, побарувачката оди подалеку. Кога корисникот ќе фати деформабилен објект со неговата рака, секој визуелен артефакт е поприметлив в од обично: тунелирање, нестабилност, тресење или нереалистично деформирање сите ја кршат илузијата. Виртуелната реалност ги обновува интересите за робусни техники за детекција на колизија токму заради тоа што грешките се толку видливи, некоја грешка што корисникот би ја толерирал на монитор станува проблем кога неговата рака интерактира со објектот. Во медицинско и хируршко тренирање пак, меките тела не се специјален ефект туку се главна грижа. Хируршките симулатори (Simbionix, Mimic Technologies, академски системи од SOFA) имаат потреба од ре-

листична симулација на меки ткива, крвни садови и кожа кога се под контакт на хируршки алати. Кај нив грешните симулации може да го нарушат тренингот и да направат погрешни навики кај ученикот.

Во анимацијата, филмовите и специјалните ефекти, каде што најчесто симулациите не се извршуваат во реално време се применуваат интерактивни симулации на меки тела во фазите на предпродукција. Houdini, Blender и SideFX имаат симулации кои ја замаглуваат границата помеѓу предпродукциски интерактивни симулации и финалниот продукт. Во интернет продавниците и виртуелни системи за испробување облека симулаторите треба да „облечат“ произволно 3Д човечко тело со ткаенини што точно се спуштаат на телото. Ова е прилично нова област во која се користат СМТ (симулации на меки тела).

1.2 Проблемот на колизии за деформируеми тела

За да се разбере зошто проблемот на колизија на меки тела е потежок од колизијата кај тврдите тела, корисно е да се почне од тврдите тела. Тврдиот објект има фиксна геометрија: кутија, топка, мрежа од триаголници итн. Проверките за растојание до геометријата може да се пресмета однапред (со Signed Distance Fields, BVH или GJK/EPA) и контактот помеѓу две тврди тела се сведува на манипулирање на конечни контактни многообразија(mainfolds)??- најчесто точки, рабови или страни. Геометријата не се изменува во векторскиот простор на телото, па било која структура за забрзување која што се гради еднаш останува валидна низ целиот век на симулацијата. Децениите истражувања имаат произведено мноштво оптимизирани алгоритми за решавање на овој проблем.

Меките тела од друга страна ги прекршуваат сите претпоставки врз кои се потпираат техниките за колизии на тврди тела. Во остатокот од овој дел ќе се набројат главните проблеми, а главниот дел од дипломската работа (5та и 6та глава) ќе се фокусира на техниките кои се користат за да се решат проблемите.

Геометријата се променува на секоја слика(frame). „Обликот“ на мекото тело е моменталната конфигурација на точките од кои е составено. Тој облик се менува на секој временски чекор на симулацијата. Било која структура за пресметка на растојанието (BVH, SDF) која што е пресметана за телото мора да се изгради одново на секој временски чекор. Цената на реизградбата станува една од главните грижи, дрвото што е изградено еднаш на почеток и не се менува веќе не може да се примени.

Бројот на контакти е огромен. Репрезентацијата на меки тела која што е мрежа од стотици или илјадници точки може да има истовремен контакт со друг објект кај десетици или стотици од овие точки. Множеството на контактни точки не е само мал дел од објектот туку цело множество, па алгоритмите што лошо скалираат со бројот на контакти веднаш наоѓаат на проблеми. Ова изнудува добри избори за податочни структури и алгоритми што овозможуваат паралалелно процесирање и што се добри за кеширање.

Колизии на телото само со себе се тежок проблем. Мекото тело може

да се превитка и да се судри само со себе. Овие сопствени колизии значат дека секоја точка потенцијално може да се судри со секоја друга точка од телото, што за пресметување е $O(n^2)$ без структури за забрзување. Широката фаза на детекција на колизии треба да биде ефикасна и тела што се движат. Униформно хеширање [5] е најчесто решение и се користи во овој труд.

Тунелирање. Кога брзината на една точка во еден временски чекор е поголема од дебелината на некој објект со кој треба да се судри, може таа точка да помине низ објектот без колизијата да се детектира. Ова не е проблем уникатен за меки тела, но е почеста појава бидејќи повеќе честички се движат одеднаш, и „лесни“ честички кои добиваат големо забрзување може моментално да имаат високи брзини. Континуирана детекција на колизии (CCD) го решава овој проблем но со голема цена на пресметка, во овој труд се користи само дискретна детекција и се спомнува ова ограничување.

Сите овие проблеми се генерално *ортогонални*. Решавањето на еден од нив не ги решава останатите. Точен систем за симулација на меки тела треба да има множество од алатки кои работат заедно наместо само еден алгоритам. Мапирањето на тие алатки е целта на овој труд.

1.3 Цели и преглед на трудот

Една цел на овој труд е да се проучат главните техники за симулирање на колизии во модерните симулатори на меки тела, забрзување на широк појас, забрзување на тесен појас и реакции на колизии. Втората цел е да се демонстрира имплементација на овие техники во симулатор на меки тела во реално време базиран на XPBD и напишан во C++.

Остатокот од документот е структуриран на следниот начин:

Глава 2 (Позадина и поврзани трудови) дава општ преглед на методите на симулација на меки тела: системи на маса и еластична пружина, методи на конечни елементи, динамика базирана на позиции. Потоа ги проучува пристапите кои се користат во постоечките физички симулатори и трудовите на кои се базира овој дипломски труд.

Глава 3 (Математички основи) ги претставува математичките методи: Њутново движење и Ојлерова интеграција, динамика базирана на ограничувања (constraint-based dynamics) со Лагранжови множители, формулацијата на XPBD (extended position-based dynamics). Исто така се воведуваат геометриските примитиви што се појавуваат во останатите глави.

Глава 4 (Модел на меко тело) ја објаснува дискретизацијата: податочната структура што се користи, методот за тетраедрализација на објектот, разликата помеѓу внатрешни и надворешни честички и формулацијата на ограничувањата за рабови и волумен.

Глава 5 (Техники за детекција на колизии) е главниот дел. Се претставуваат забрзување во широка фаза (BVH и просторно кеширање), забрзување на тесна фаза

кај мрежи од триаголници(најблиска точка, псевдо-нормали) и сопствени колизии. Секоја подглава алтернира помеѓу генералната техника и конкретната имплементација.

Глава 6 (Техники за реакција на колизии) ги претставува методите за придвижување на мекото тело откако е детектирана колизија.

Глава 7 (Имплементација и рендерирање) ја покрива имплементацијата во код на симулаторот: структурата на проектот, rendering pipeline, контроли со корисничкиот интерфејс.

Глава 8 (Резултати и евалуација) претставува тест сценарија и стрес тестирање на симулацијата.

Глава 9 (Заклучок) се сумаризира направеното и се дава осврт на идни подобрувања: непрекинато детектирање на колизии, колизии помеѓу 2 меки тела, паралелизација, додавање материјали.

2. Позадина и поврзани дела

Литературата за симулација на меки тела има произведено повеќе различни фамилии на методи. Тие се разликуваат во начинот на кој го дискретизираат телото, како ги поврзуваат силите со деформацијата и како интегрираат со текот на времето. Разбирањето на нивните предности и недостатоци е неопходно пред да се дискутираат колизиите, бидејќи различни техники за колизии функционираат добро за различни парадигми

2.1 Парадигми за симулација на меки тела во реално време

2.1.1 Системи на маса и пружина

Тие се најраните и наједноставните модели. Телото е дискретизирано како множество од точкести маси (point masses) кои се поврзани со Хукови пружини; силите на пружината го почитуваат Хуковиот закон и се интегрираат напред во времето, најчесто со експлицитна Ојлерова или Верле интеграција. Прово [1] го пионира овој пристап за ткаенина, со структурни, смолкнувачки и искривувачки пружини што ја формираат основата на моделот. Бараф и Виткин [2] го прошируваат моделот со имплицитна интеграција за да се премине пречката на експлицитна стабилност.

Јачината на овие системи е во нивната едноставност: имаат едноставен математички модел и податочните структури се тривијални. Слабостите се тоа што цврстината (stiffness) заедно со експлицитна интеграција произведува нестабилност, што форсира користење на мали временски чекори. Дополнително, однесувањето е анизотропично зависно од топологијата, изборот на триангулација на ткаенината има ефект на тоа како таа се однесува. Подесувањето на систем на маса и пружина за да соодветствува на вистински материјал е многу тешко. И покрај овие недостатоци, овој модел се уште е чест во симулацијата на ткаенина и во едукативни имплементации токму поради тоа што е едноставен.

2.1.2 Метод на конечни елементи (Finite Element Method)

Методот на конечни елементи (понатаму FEM) е главниот пристап на механика на континуум. Телото се дискретизира во елементи (тетраедри во 3Д, триаголници во 2Д) и симулацијата решава за полето на изместување (displacement field) што минимизира функција на енергија што се изведува од напрегањето. Има 3 фамилии кои се важни за меки тела:

Линеарен FEM користи линеарна(Коши) мерка за напрегање. Тој е брз бидејќи може да се изврши предпресметка на повеќето елементи од почетната конфигурација. Проблемот е што не е ротационо инваријантен овој метод, тело што се ротира а нема деформација ќе регистрира како да нема напрегање, што произведува невидливи сили што го туркаат телото во погрешни насоки.

Коротациски FEM (Милер и Грос [6]) го решава проблемот на ротација со екстрахирање на ротација по елемент од градиентот на деформација и аплицирање на линеарно напрегање во ротираната рамка. Резултатот е ротационо инваријантен и стабилен за деформации од нормална голема, со средно покачување на цената за пресметка. Коротацискиот FEM е де факто стандард за FEM меки тела во реално време во интерактивни апликации.

Нелинеарен FEM користи напрегање од Green или други видови на мерења за напрегање заедно со хипереластична густина на енергија(Нео-Хукова, Муни-Ривлин). Методот е физички точен, но скап: градиентот е покомплексен и стабилната интеграција најчесто бара имплицитни методи кои решаваат линеарен или нелинеарен систем на секој временски чекор.

2.1.3 Спојување на облици (Shape matching)

Спојувањето на облици [6] е метод без мрежи (meshless method). Деформираната конфигурација на кластер од честички се спојува со конфигурацијата во мирување со пресметување на цврстата трансформација со најдобро прилагодување (best fit), по што честичките се повлекуваат кон трансформираниите почетни позиции. Методот е ефтин, многу стабилен и елегантно се справува со големи деформации и ротации. Главното ограничување е тоа што контролира глобален облик на телото наместо локални напрегања, што прави мали одлики на телото тешко да се прикажат. NVIDIA Flex користи спојување на облици за цврсти тела.

2.1.4 Динамика базирана на позиции (Position-Based Dynamics)

Динамика базирана на позиции (понатаму PBD)[3] има различен пристап: наместо да се изведуваат силите од енергијата и да се интегрира, директно проектира честички за да се задоволат геометриски ограничувања. После чекор на предвидување на позиција во симплектичен-Ојлер стил (Symplectic-Euler), итеративен Гаус-Сајдел решавач го посетува секое ограничување (растојание, волумен, виткање, колизии) и ги корегира позициите на честичките за да се задоволи ограничувањето. Брзините на честичките се добиваат од промената на позицијата на крајот на временскиот чекор.

Резултатот е безусловно стабилен и лесен да се надгради со додатни типови на ограничувања. PBD доминира во симулациите на меки тела во реално време во игри и виртуелната реалност, делумно бидејќи бројот на итерации дава интуитивна контрола на квалитетот и делумно бидејќи формулациите базирани на ограничувања за ткаенини, јажиња, коса, течности и грануларни материјали сите може да се вклопат во еден решавач.

Главното ограничување на обичниот PBD е што ефективната цврстина зависи од бројот на итерации: удвојувањето на итерациите ефективно ја удвојува и цврстината на телата. Ова прави потешкотии при специфицирање на конкретен материјал, и корисникот мора да ги подесува итерациите и временскиот чекор заедно за да ја добие посакуваната цврстина.

2.1.5 Проширена диманика базирана на позиции (Extended PBD)

Extended PBD (понатаму XPBD) го реформулира PBD како ограничен оптимизациски проблем со усогласеност (compliance), при што акумулира Лагранжови множителни низ итерациите. Параметарот за усогласеност α означува единица на инверзна цврстина, а во ажурирањето на позиции во секоја итерација се користи $\tilde{\alpha} = \frac{\alpha}{dt^2}$, што ја прави цврстината на телото независна од бројот на итерации.

XPBD ја задржува стабилноста на PBD и можноста за надоградување при што се додава параметар за полесно специфицирање на специфични материјали. Тој е интеграторот што се користи во овој труд и формулацијата се објаснува формално во глава 3.

Paradigm	Cost	Stability under stiffness	Material control	Typical use
Mass-spring	Very low	Poor with explicit integration	Coarse, topology-dependent	Cloth, education
Linear FEM	Low	Moderate	Continuum strain	Small deformations
Corotational FEM	Moderate	Good	Continuum strain, rotation-invariant	Real-time soft bodies, surgical sim
Nonlinear FEM	High	Good (with implicit)	Hyperelastic	Offline accurate sim
Shape matching	Very low	Excellent	Global, not local	Stylised solids, Flex
PBD	Low	Excellent	Compliance, iteration-coupled	Real-time everything
XPBD	Low	Excellent	Compliance, iteration-independent	Real-time everything (preferred)

Табела 2.1: Comparison of simulation paradigms

2.2 Историја на методи базирани на позиција

Методите базирани на позиција сега се широко прифатени во графиката во реално време, но имале инкрементален развој низ годините. Секој од овие чекори решавал

проблеми од претходните трудови.

Прово[1] користи модел на маса и пружина но додава чекор што прави корекција на рабовите кои се преиздолжени и ги враќа на некоја максимална должина, што ефективно додава ограничувања на должина. Ова е концептуалниот почеток на методите базирани на позиција: наместо да се пресмета само сила на пружина, директно се спроведува дистанцата.

Бараф и Виткин[2] прават имплицитна интеграција на равенките на движење на ткаенини, со што дозволуваат поголеми временски чекори од експлицитните методи. Тука се поставува проблемот на стабилност наспроти цврстина што подоцна PBD целосно го одбегнува.

Јакобсен[7] на неговата GDC презентација за комплексна физика на карактери објаснува за Верле интеграција споена со ограничувања за растојание применети итеративно. Било многу влијателно за игрите, многу независни играчки погони ги имаат изградено своите системи за ткаенини врз основа на оваа рамка пред терминот PBD да постои.

Милер, Хејделбергер, Тешнер[8] го воведуваат методот на спојување на облици спомнат погоре што подоцна се интегрира во PBD системите како еден тип на ограничувања.

Милер, Хајделбергер, Хеникс, Ратклиф[3] го воведуваат PBD спомнат погоре што со огромна брзина почнува да се користи во игри и во академијата.

Маклин и Милер[9] со *Position Based Fluids*. Додаваат ограничување на густина врз честички, што создава однесување на течност налик на SPH во рамка на PBD. Демонстрираат дека формулацијата базирана на ограничувања генерализира надвор од меки тела и ткаенини.

Маклин, Милер, Чентанез[4] со XPBD ја воведуваат формулацијата објаснета погоре и што се користи во овој труд.

После 2017 се намалува брзината на фундаментални промени, понатамошните трудови се фокусираат за додавање на екстензии(тетиви, анизотропни материјали, течности) и инженерски подобрувања(имплементации на GPU, паралелно распоредување на ограничувања, справување со контакти). XPBD останува најкористената формулација за користење во системи во реално време во моментот на пишување на овој труд.

2.3 Техники на колизии во физички симулации

Колизиите во физичките симулации имаат своја литература долга децении што е главно независна од литературата за деформации. Спојувањето на двете е главна цел на овој труд.

2.3.1 Тврди наспроти меки тела

Колизија на цврсти тела е добро проучен проблем. За конвексни тела, GJK (Gilbert, Johnson, Keerthi) и EPA(Expanding Polytope Algorithm) пресметуваат

растојание и длабочина на пробивање. За произволни триаголнести мрежи, BVH и SAT (Separating Axis Theorem) се стандардот. Контактот помеѓу две цврсти тела се редуцира на мало множество на контактни точки со нормали и длабочини.

Колизијата на меки тела има поголеми побарувања. Геометријата на телото се менува на секоја слика, па структурите за забрзување мора да поддржуваат ре-изградба или нагудување. Множеството на контакт е огромно: стотици честички може да се во контакт истовремено. Сопствените колизии исто така се од големо значење, телото може да се допира самото себе и широката фаза на забрзување мора добро да се справи со овој проблем.

2.3.2 Дискретна наспроти континуирана детекција

Дискретните детекции за колизии ги проверуваат позициите на крајот на временскиот чекор, доколку честичката се најде внатре во друго тело, се генерира контакт. Ова е ефтино за проверка и се користи во овој труд. Цената за ова е *тунелирање*, ако честичката се движи доволно брзо може да пројде низ некое доволно тенко тело без да се детектира колизија.

Континуирана детекција на колизии (оттука CCD) го тестира целиот волумен што го опишува честичката при своето движење во временскиот чекор, доколку се сече со некое друго тело се генерираат колизии што би се пропуштиле при дискретната алтернатива. CCD е точно но скапо, најчесто неколу пати поскапо за секоја проверка од дискретна проверка. Модерните игри користат комбинација од дискретни со селективни континуирани детекции за некои објекти

2.3.3 Парадигми за реакција на колизија

Постојат четири главни семејства на реакција при судир, при што секое има свои предности и недостатоци.

Методите со казни [10] го третираат судирот како цврста пружина: силата на реакција е пропорционална на длабочината на продирањето, придвижувајќи ги телата што се судираат настрана. Едноставни се за имплементирање, но цврстината на пружината станува параметар за подесување што се бори против стабилноста на интеграцијата. Методите со казни се во голема мера застарени во современите енџини за меки тела, преживувајќи главно во хаптички симулатори каде што формулацијата со "цврста пружина" се совпаѓа со моделот на хаптичка сила.

Импулсните методи [11] пресметуваат моментална промена на брзината во точката на контакт што спречува меѓусебно продирање, користејќи ги законите за зачувување на контактната механика. Стандарди се за симулатори за цврсти тела (Bullet, ODE, Box2D), каде што равенките за цврсти тела допуштаат импулсни решенија во затворена форма. Помалку се применуваат за меки тела бидејќи претпоставката за "цврстина" се нарушува при секој контакт.

Одговор заснован на ограничувања [2] [12] ја третира непродорливоста како ограничување на нееднаквост и решава линеарен комплементарски проблем (LCP) или проектирана Гаус-Сајдел итерација. Точен, добро се справува со натрупување и

триење, но е скап и сложен за имплементација.

Проекцијата на положбата едноставно ги поместува навлезните честички назад на површината, а потоа ја коригира брзината за да се усогласи. Наједноставниот можен одговор — природен во PBD/XPBD бидејќи интеграторот е заснован на позиции. Ова е техниката што се користи во овој труд.

Поновата насока е судирите да се формулираат како XPBD ограничувања со сопствени Лагранж множители и усогласеност, со што се интегрира ракувањето со контакт во самиот решавач на ограничувања (Милер[13]). Ова е природен следен чекор надвор од пристапот со проекција на положба што се користи овде, и е дискутиран како идна работа во Поглавје 9.

2.3.4 Пристапи за само-судир

Само-судирот е потежок од судир тело со тело бидејќи телото е подвижен облак од точки, а не фиксен судирач. Тука доминираат три пристапи.

Просторно хеширање (Тешнер и сор. 2003[5]). Има униформна мрежа каде што секоја честичка се хашира во ќелија и се тестираат парови честички во истата или во соседните ќелии. Едноставно, добро прилагодено за деформни тела.

BVN од геометријата што се деформира. BVN изграден еднаш во мирување и се прилагодува во секоја слика на деформираната конфигурација. Ефикасно кога деформацијата е умерена, вообичаено во симулатори за ткаенина. Ларсон и Акенин-Молер [14] воведоа прилагодување од дното нагоре.

Континуирана само-колизија за ткаенина[12]. CCD за раб-на-раб и точка-триаголник специјално прилагодено за ткаенина, вклучувајќи зони на удар за справување со кластери што се сечат. Стандард во офлајн симулатори за ткаенина.

Изборот помеѓу нив зависи од тоа дали симулаторот е наменет за дискретно или континуирано детектирање, топологијата на телото (ткаенина наспроти волуменска) и буџетот по слика.

3. Математички и физички основи

Во овој дел ќе се објаснат математичките и физичките основи кои се користат во останатите глави.

3.1 Њутново движење и временско интегрирање

Почетната точка е вториот Њутнов закон за секоја честичка:

$$m_i \cdot a_i = F_i$$

каде што F_i е вкупната сила на честичката i . Во овој труд, единствената надворешна сила е гравитацијата, $F_{grav} = m_i \cdot g$, така што интеграторот е одговорен само за напредување на позициите и брзините под дејство на гравитацијата; сите други влијанија (ограничувања на рабови, ограничувања на волумен, судири) се воведуваат како ограничувања во §3.2 и се проектираат за време на решавањето на ограничувањата.

Дискретизацијата во времето создава нумерички интегратор. Три се релевантни.

3.1.1 Експлицитен (forward) Ојлер

Тој е наједноставната шема:

$$\begin{aligned}v(t + \Delta t) &= v(t) + \Delta t \cdot \frac{F(t)}{m} \\x(t + \Delta t) &= x(t) + \Delta t \cdot v(t)\end{aligned}$$

Експлицитниот Ојлер е точен до прв ред (грешката се намалува линеарно со бројот на итерации) и многу ефтин, но два проблема го прават несоодветен како примарен интегратор за симулатор на честички. Прво, тој не ја зачувува енергијата: вкупната енергија на затворен конзервативен систем расте со текот на долготото симулирање. Едноставен хармониски осцилатор симулиран со експлицитниот Ојлер покажува видливо спирално однесување во рок од неколку стотици временски чекори. Второ, тој е *нестабилан за тврди системи*: максималниот стабилен

временски чекор се скалира како $\Delta t \leq 2/\omega$, каде што ω е највисоката природна фреквенција во системот. Тврдите ограничувања ја зголемуваат ω , принудувајќи на многу мали временски чекори.

3.1.2 Симплектички(полуимплицитен) Ојлеров метод

Се прави мала модификација на експлицитниот Ојлеров метод:

$$\begin{aligned}v(t + \Delta t) &= v(t) + \Delta t \cdot \frac{F(t)}{m} \\x(t + \Delta t) &= x(t) + \Delta t \cdot v(t + \Delta t)\end{aligned}$$

Клучната промена е што ажурирањето на позицијата го користи ажурираниот вектор на брзина. Ова го прави методот симплектички интегратор: тој зачувува дискретна апроксимација на енергијата (и други Хамилтонови инваријанти) на долги временски опсези. Поместувањето на енергијата е ограничено, наместо мононо.

Симплектичкиот Ојлер е чекорот за предвидување што се користи во PBD и XPBD. Во комбинација со проекција на ограничувањата во чекорот за пресметување на позицијата, тој дава безусловно стабилно однесување дури и за тврди материјали, бидејќи интеграторот не мора директно да ракува со тврдите сили, тој само треба да ги предвиди позициите, кои потоа ги коригира решавачот на ограничувања.

3.1.3 Верлеова интеграција

Трета вообичаена шема е Верлеовата интеграција:

$$x(t + \Delta t) = 2x(t) - x(t - \Delta t) + \Delta t^2 \cdot \frac{F(t)}{m}$$

Со Верле методот не се складира експлицитна брзина; брзината се пресметува од разликата помеѓу последователните позиции. Верле е исто така симплектички и е математички еквивалентен на симплектичкиот Ојлер за системи со константна маса и конзервативни сили. Презентацијата на Јакобсен на GDC во 2001 година го популаризираше Верле во игрите токму затоа што ограничувањата можат да се применат директно на позициите без потреба од следење на брзините, став што PBD го наследи.

Во современите формулации, изборот помеѓу симплектичкиот Ојлер и Верле е во голема мера козметички. Оваа теза ја следи XPBD формулацијата [4] во користењето на симплектичкиот Еулер со експлицитно следење на брзината, така што пригушувањето на брзината и триењето се полесни за изразување.

3.2 Динамика заснована на ограничувања (Constraint-based dynamics)

Динамиката заснована на ограничувања е реформација на проблемот со симулација. Наместо да се специфицираат сили и да се врши интегрирање, се специфицира што мора да важи за конфигурацијата: зачувана должина на рабови, зачуван волумен, непенетрирање на честичките и се дозволува решавачот да ја пресмета корекцијата на положбата што ги прави тие тврдења точни.

3.2.1 Ограничувања

Ограничување е скаларна функција $C(X)$ на конфигурацијата на системот. Во оваа теза се појавуваат два вида:

- *Ограничувања на еднаквост:* $C(X) = 0$. Примери: должината на работ е еднаква на нејзината должина во мирување; волуменот на тетраедар е еднаков на неговиот волумен во мирување.
- *Ограничувања на нееднаквост:* $C(X) \geq 0$. Пример: честичката мора да остане надвор од судирачот, така што $C(X) = \text{означена оддалеченост до површината}$.

Градиентот на ограничувањето $\nabla C \in \mathbb{R}^{3n}$ ја означува насоката во просторот на конфигурации во која C се менува најбрзо. Повеќето градиенти на ограничувањата се ретки — ограничувањето на работ помеѓу честичките i и j има вредности различни од нула само во блоковите што соодветствуваат на x_i и x_j .

3.2.2 Лагранжови множители

Третирајќи го C како холономско ограничување, силата на ограничувањето ја има формата $f = \lambda \nabla C$, каде што λ е скаларен Лагранжов множител. Интуицијата е дека ограничувањето го турка системот по насока на ∇C за количина пропорционална на тоа колку е моментално прекршено. Множителот се одредува со барањето системот да остане на множеството на ограничување $C = 0$.

Со едно ограничување и предвидената положба x' од 3.1, проектираната положба x која го задоволува условот $C = 0$ (во прв ред) е:

$$\Delta x = M^{-1} \nabla C \lambda$$
$$\lambda = -\frac{C(\vec{x})}{\nabla C \cdot M^{-1} \cdot \nabla C}$$

Ова е линеаризирана проекција на множеството на ограничувања, со тежина од инверзната маса. Истата идеја се генерализира на повеќе ограничувања.

3.2.3 Гаус-Сајделова итерација

Систем од m ограничувања генерира поврзан систем на равенки за множителите. Решавачите од типот PBD не го решаваат поврзаниот систем директно; наместо тоа, тие ја извршуваат Гаус-Сајдел итерацијата, посетувајќи го секое ограничување

по ред, независно проектирајќи го и ажурирајќи ја позицијата. Следното ограничување ја гледа ажурираната позиција, така што поврзаноста е фатена имплицитно преку редоследот на евалуација.

Гаус-Сајделова итерација конвергира релативно бавно во споредба со глобалните решавачи (на пр. конјугиран градиент на линеаризираниот систем), но е тривијална за имплементирање, не бара глобално собирање и е лесна за стабилизирање. Тоа е стандарден образец на решач во PBD/XPBD во реално време. Тековната имплементација стандардно користи десет Гаус-Сајделови итерации по потчекор; ова е прикажано како лизгач во графичкиот кориснички интерфејс (GUI).

3.3 XPBD формулацијата

XPBD го дополнува PBD со две идеи: подложност и акумулирани множители. И двете се неопходни за цврстината да биде независна од бројот на итерации. Овој дел ги претставува равенките за ажурирање на XPBD на кои се потпира остатокот од тезата.

3.3.1 Усогласеност (compliance)

Усогласено ограничување е ограничување со конечна крутост. Усогласеноста α е обратното од крутоста k ; $\alpha = 0$ одговара на тврдо ограничување ($k = \infty$), а $\alpha > 0$ на меко ограничување. Специфицирањето на материјал во однос на усогласеност е попогодно отколку специфицирањето на крутост, бидејќи $\alpha = 0$ е добро дефинирано, додека ($k = \infty$) не е.

Во имплементацијата, и рабовните и волуменските ограничувања имаат изложен параметар на подложност (кој се поставува преку графичкиот кориснички интерфејс) кој се појавува во ажурирањето по итерација, изведено подолу.

3.3.2 Терминот alpha

Кога XPBD се изведува од имплицитна временска интеграција на системот на ограничувања со усогласување, усогласеноста се појавува во равенките за ажурирање поделена со Δt^2 :

$$\tilde{\alpha} = \frac{\alpha}{\Delta t^2}$$

Ова деление одразува дека усогласеноста се компензира со чекорот на времето: ограничување со усогласеност α се однесува поцврсто при помали чекори на времето. Терминот $\tilde{\alpha}$ е она што ја прави цврстината на XPBD независна од итерацијата — тој се појавува експлицитно во ажурирањето по итерација, така што решавачот ја знае ”вистинската” целна усогласеност без оглед на тоа колку итерации се потребни.

3.3.3 Акумулиран Лагранж мултипликатор

Во PBD проекцијата во секоја итерација е независна; множителот од претходната итерација се заборава. Во XPBD множителот λ се задржува низ итерациите во рамките на потчекор, собирајќи го вкупниот импулс на ограничувањето што е применет. λ се ресетира на нула на почетокот на секој потчекор и на секоја итерација се решава само за прирастот $\Delta\lambda$.

Оваа разлика е она што го прави XPBD да конвергира кон одредена цел, наместо да осцилира околу неа.

3.3.4 Ажурирањето по итерација

За едно ограничување C со градиент ∇C и акумулиран множител λ , една итерација на XPBD пресметува:

$$\Delta\lambda = -\frac{C(X) - \tilde{\alpha}\lambda}{(\nabla C(x) \cdot M^{-1} \cdot \nabla C(x)^T + \tilde{\alpha})}$$

Читајќи го ажурирањето ред по ред:

Броителот $(C + \tilde{\alpha}\lambda)$ е тековното пречекорување на ограничувањето зголемена за терминот на усогласеност. Како што λ расте, зголеменото се намалува, анализирајќи дека ограничувањето се приближува кон задоволување. Именителот $(\nabla C(x) \cdot M^{-1} \cdot \nabla C(x)^T + \tilde{\alpha})$ е генерализираната инверзна маса (или "ефективна маса") на ограничувањето, плус $\tilde{\alpha}$. Додаденото $\tilde{\alpha}$ го регулира системот кога геометријата е лошо определена и ја претвора проекцијата од бесконечно цврста во усогласена. Зголемувањето на положбата ∇X е во насока на $M^{-1}\nabla C$, скалирано со $\Delta\lambda$. Потешките честички (со помали w_i) се движат помалку; полесните честички се движат повеќе. За $\alpha = 0$, ажурирањето точно се сведува на PBD проекцијата — XPBD е строга генерализација на PBD.

3.3.5 Ажурирање на брзината

Откако сите ограничувања ќе се итерираат до конвергенција (или ќе се потроши буџетот за итерации), брзините се пресметуваат од промената на позицијата во текот на потчекорот:

$$v = \frac{x - x_{prev}}{\Delta t}$$

Ова имплицитно ажурирање на брзината е карактеристична особина на методите засновани на позиција: силите никогаш не се појавуваат експлицитно во решаваачот на ограничувања, туку само позициите и брзините изведени од нив.

3.3.6 Зошто функционира цврстината независна од итерацијата

Интуицијата за цврстината независна од итерацијата: зголеменото прекршување $(C + \tilde{\alpha}\lambda)$ мери колку е системот далеку од целната конфигурација со цврстина, а не од конфигурацијата со строго ограничување. Како што се зголемува λ , системот се опушта кон рамнотежа со брзина одредена само од $\tilde{\alpha}$, а не од бројот на итерации.

Дуплирањето на итерациите го приближува системот до истата цел, наместо да ја поместува самата цел.

3.3.7 Потчекори наспроти итерации

Трудот на Macklin од 2019 година, "Small Steps in Physics Simulation", тврди дека, со даден фиксен вкупен пресметковен буџет, повеќе потчекори со помалку итерации во секој од нив генерално даваат подобри резултати отколку помалку потчекори со многу итерации. Вообичаена конфигурација е 10 потчекори $\times$ 1 итерација, наместо 1 потчекор $\times$ 10 итерации. Имплементацијата ги изложува и двете како параметри; стандардната вредност е 10×10 (цел од 60 fps со удобна маргина).

4. Модел на меко тело

Кога се импортира некоја мрежа за рендерирање на екран (пример од .obj датотека), во основа се добиваат две листи: листа на 3Д темиња и листа на триаголници кои ги индексираат тие темиња. Тоа е доволно да се исцрта објектот (бидејќи рендерерот ги растеризира триаголниците), но тоа само ја опишува површината на објектот. За рендерирање ова е доволно, но за физички симулации не е.

4.0.1 Зошто празна обвивка не може да симулира меко тело

Како пример можеме да земеме мрежа што репрезентира сфера. Ако наместиме ограничувања за растојание помеѓу соседните темиња (што се рабовите на телото) и со тоа пробаме да симулираме меко тело, добиваме 2 проблеми:

- **Нема задржување на волумен.** Ако се турне едно теме навнатре и системот за ограничувања на телото нема нешто што ќе се спротистави на движењето. Другите темиња остануваат таму каде што се, само рабовите што се директно истегнати или компресирани ја регистрираат промената. Поради ова телото се сплескува како издишан балон кога има било каква сила што турка навнатре
- **Инверзиите се невидливи.** Ако мрежата се извитка низ себеси (надворешна страна пројде низ внатрешна), рабовите на површината остануваат со иста должина и ограничувањата не може да разликуваат дали телото се извртело однадвор навнатре.

Обвивката енкодира дека „овој триаголник се наоѓа тука“, но не „овој регион е цврст“. Мекото тело има потреба од второто

4.0.2 Тетраедрализација (tetrahedralization)

Стандардниот начин да се реши проблемот е да се наполни внатрешноста на телото со густо множество на тридимензионални ќелии кои имаат волумен кој може да се измери. На тие волумени може да се дадат ограничувања (да се враќаат кон волуменот во мирување) и телото станува отпорно на компресија навнатре.

Природниот геометриски примитив за ова е **тетраедарот**, пирамида со 4 темиња што е наједноставното тридимензионално тело со волумен што не е 0 и е добро дефиниран. Тетраедрите имаат неколку особини што ги прават соодветни:

- **Наједноставен примитив.** Три точки дефинираат триаголник, а четири тетраедар. Било што поедноставно е дегенеративно
- **Волумен во затворена форма.** Волуменот е лесен за пресметување со користење на тројниот продукт на три вектори. Нема интеграција и може да се диференцира
- **Едноставни градиенти.** Градиентот на волуменот во однос на секое теме е векторски производ на два спротивни рабови.
- **Има постоечки алатки за пресметка.** Библиотеки како TetGen и CGAL добиваат мрежа од точки како влез и даваат тетраедрализирано тело во секунди

4.0.3 Делови на мекото тело

Откако се прави тетраедрализација, мекото тело има четири дела, од кои секој е мотивиран од специфична потреба:

- Површински честички (pos, vel, invMass). Телото е облак од точки - нема површина или волумен сè додека ограничувањата не ги декларираат.
- Внатрешни честички. Точки кои се додаваат во внатрешноста на телото при тетраедрализација. Рендерерот не ги користи, тие постојат само за да се пресметаат волуменските ограничувања. Сите имаат иста структура како внатрешните точки (позиција, брзина и инверзна маса).
- Рабни ограничувања (едно по уникатен раб на површината). Секое ограничување чува пар на точки со нивната далечина при мирување. Овие ограничувања се спротиставуваат на истегнување и ја дефинираат надворешната форма на објектот.
- Волуменски ограничувања. Има едно за секој тетраедар што се добива при тетраедрализација. Секое содржи волумен на тетраедарот при мирување. Овие ограничувањата се спротиставуваат на компресија и инверзија.

Двете ограничувања имаат различна улога: рабовите ја задржуваат надворешната форма, волуменот ја задржува внатрешната. Рабовите сами прават обвивка што се истегнува, а волумените само прават множество тетраедри без обвивка. Тие заедно даваат тело што може да се деформира.

4.1 Репрезентација на честички

Секоја честичка содржи три информации за состојбата: позиција $x \in \mathbb{R}^3$, брзина $v \in \mathbb{R}^3$, и инверзна маса $w = \frac{1}{m}$. Тие се складираат во **std::vector** низи во **SoftBody struct** во include/physics/particle.hpp.

Одлуката да се користи *инверзна* маса е намерна и соодветствува на конвенцијата на PBD/XBPD. Секое ажурирање на ограничувањата дели со сума на инверзни маси, па кеширање на w избегнува делење на секоја итерација. Поважно, $w = 0$ е природно енкодирање на неподвижна честичка: секоја корекција $\Delta x = w \Delta \lambda \nabla C$

која се применува врз честичката дава нула, па таа останува во иста положба без додавање услови во решавачот.

Втор пар на низи, *initialPos* и *initialVel* се зачувуваат при градење на објектот. Тие се почетната конфигурација и се користат за ресетирање на симулацијата преку графичкиот интерфејс без одново да се гради објектот.

Топологијата се чува одвоено од состојбата на секоја честичка:

- **tets** - листа од `array<int, 4>` четворки на индекси, една по тетраедар од волуменската мрежа
- **surfaceIndices** - листа од индексите на темињата од кои се состои мрежата на површинските честички. Се користи од рендерерот за приказ на телото и за ажурирање на позициите на секоја слика
- **numSurfaceParticles** - бројот на уникатни темиња на површината. Се користи од рендерерот за да се испртаат точниот број на темиња.

Распределба на маса: масата по честичка е волуменски нормализирана, при што масата од тетраедрите се префрла на нивните темиња. Секој тетраедар со волумен V придонесува со $\frac{V}{4}$ од волуменот на телото во секое од своите четири темиња, па акумулираната вредност за честичката е вкупниот волумен на материјал што таа го претставува. Аргументот за маса во конструкторот е посакуваната вкупна маса на телото; еднакво распределена густина $\rho = \frac{mass}{V_{total}}$ го претвора акумулираниот волумен во маса по честичка, давајќи $w_i = \frac{1}{(\rho \cdot V_i)}$. Ова е пофизичко од еднакви маси, честичките во поголемите или погустите региони на тетраедрите имаат поголема инерција, додека честичките во тенките региони се полесни, и ја одржува вкупната маса на телото независно од скалата на мрежата. Волуменот на секој тетраедар го реупотребува скаларниот троен произвол кој подоцна се користи од страна на ограничувањето за волумен (§4.4). Честичките кои не припаѓаат на ниту еден тетраедар, и дегенерирана мрежа со вкупен волумен нула, се враќаат на $w = 0$ (прицврстени), во согласност со конвенцијата за прицврстување со инверзна маса погоре.

4.2 Тетраедрализација

4.2.1 TetGen

Волуменската мрежа во симулатор е произведена од TetGen [15], генератор на тетраедрални мрежи базиран на Делоне метод. TetGen на влез зема PLC (piecewise linear complex) и како резултат дава ограничена Делоне тетраедрализација: се додаваат внатрешни Штајнер точки по потреба и волуменот помеѓу внатрешните страни се пополнува со тетраедри што го задоволуваат Делоне условот на празна впишана сфера.

Во симулаторот TetGen е интегриран на следниот начин:

1. Влезните темиња се запишуваат во **in.pointlist** како рамна низа

2. Површинските триаголници се запишуваат во **in.facetlist** како страни од еден полигон
3. Се конфигурира **tetgenbehavior** со знамето **pq2.0Y**
4. **tetrahedraliza(&b, &in, &out)** го извршува алгоритмот
5. Излезот **out.pointlist**, **out.tetrahedronlist**, и **out.trifacelist** се користат од низите **pos**, **tets**, **surfaceIndices** од симулаторот

Стрингот **pq2.0Y** што се користи за конфигурација се состои од:

- **p** - *PLC мод*. Му кажува на TetGen дека влезот не е облак од точки туку полиедар опишан од темиња и страни, па само внатрешноста треба да се тетраедризира
- **q1.4** - *рафинирање на квалитет* со сооднос на радиус-раб од 1.4. Овој сооднос е опишаниот радиус на тетраедарот поделен со должината на неговиот најкраток раб. Пониски вредности произведуваат повеќе тетраедри со цена на повеќе Штајнер точки и повеќе тетраедри на излез.
- **Y** - *задржување на влезната граница*. Без ова знаме, TetGen може да внесува Штајнер точки на површинските триаголници, што ќе ја наруши претпоставката дека влезните индекси на темиња се мапираат со првите позиции во низата **pos**.

4.2.2 Ограничувања

TetGen зависи од точен влез. Ако влезната мрежа не е затворена, има пресеци сама со себе, или ако има погрешна ориентација на страните, ќе се добие мрежа со дупки, грешки при извршување или тивок неуспех каде површината е точна но внатрешноста остане празна. Доколку се добие некој погрешен резултат со нов објект додаден во сцената, погрешниот тип на објект е прво нешто што треба да се провери.

4.3 Ограничувања на рабови

Ограничувањата на рабови се „надворешната“ половина на мекото тело. Во овој дел се дава формална дефиниција, градиентот кој се решава и кратка дискусија за однесувањето и подесување.

4.3.1 Дедупликација

Површинските триаголници имаат заеднички рабови, па наивно креирање на едно ограничување по раб на триаголник би креирало дупликати на рабовите. **Solver::createConstraintFromIndices** прави дедупликација со внесување на парови на индекси од темиња и скокање на веќе постоечките парови.

4.3.2 Ограничување и градиент

Функцијата на ограничувањето е отстапувањето од должината при мирување:

$$C(x_1, x_2) = \|x_1 - x_2\| - I_0$$

Должината при мирување I_0 се пресметува при конструирање на телото од иницијалните позиции на честичките, па „обликот при мирување“ на телото е геометријата што се вчитува на почеток на симулацијата.

Градиентот е единечен вектор со насока паралелна на работ:

$$\nabla_{x_1} C = \hat{n}, \nabla_{x_2} C = -\hat{n}$$

$$\hat{n} = \frac{(x_1 - x_2)}{\|x_1 - x_2\|}$$

Градиентите имаат единечна должина, што прави броителот во XPBD да се скрати до $w_1 + w_2 + \tilde{\alpha}$, што е ефтино за пресметување.

4.3.3 Чекор во решавач

Во **Solver::solveEdgeConstraint** се имплементира ажурирањето на позициите и ограничувањата:

1. Читање на позиции и инверзни маси. Излез ако двете честички имаат бесконечни маси
2. Пресметка на $I = \|x_1 - x_2\|$. Излез ако $I < 10^{-6}$ (Дегенериран раб со недефиниран градиент)
3. Нормализирање $\hat{n} = \frac{x_1 - x_2}{I}$
4. $C = I - I_0$
5. $\tilde{\alpha} = \frac{\alpha}{\Delta t^2}$
6. $\Delta\lambda = \frac{-C - \tilde{\alpha}\lambda}{(w_1 + w_2 + \tilde{\alpha})}$
7. $\Delta x_1 = w_1 \Delta\lambda \hat{n}; \Delta x_2 = -w_2 \Delta\lambda \hat{n}$
8. $\lambda \leftarrow \lambda + \Delta\lambda$

Проверките во чекор еден и два за ран излез се неопходни. Без нив, две честички кои се неподвижни ќе предизвикаат делење $0/0$, а целосно дегенериран раб ќе произведе недефинирана насока

4.3.4 Ограничување на површински рабови

Само рабовите од надворешната обвивка на телото имаат ограничувања. Внатрешните кои ги поврзуваат Штајнер точките до нивните соседи немаат ограничување на раб, тие се контролираат само од волуменските ограничувања. Друга опција е да се додадат ограничувања за секој раб во тетраедрализираната мрежа, но тоа ќе произведе поцврсто тело што поагресивно конвергира при истегнување, но истовремено ќе се додадат неколкупати повеќе ограничувања.

4.3.5 Однесување и нагудување

Со $\alpha = 0$ (или многу мала вредност), ограничувањата на раб се однесуваат скоро како цврсти ограничувања на растојание, при што телото се спротиставува на истегнување. Со поголемо α , рабовите стануваат како гума, телото повеќе се сплескува под гравитација, и осцилациите од висока фреквенција брзо се намалуваат. Бидејќи усогласеноста влага во именителот, поголемо α го прави системот постабилен.

4.4 Волуменски ограничувања

Волуменски ограничувања се „внатрешната“ половина на мекото тело, тие предизвикуваат тоа да се спротиставува на компресија и инверзија, што рабовните ограничувања не можат да го направат. Во овој дел се опишува функцијата на ограничување, формулите за градиенти, чекорот во решавачот.

4.4.1 Скаларен троен производ

Волуменот на тетраедар со темиња $\mathbf{x}_1, \mathbf{x}_2, \mathbf{x}_3, \mathbf{x}_4$ е:

$$V = \frac{1}{6}(\mathbf{x}_2 - \mathbf{x}_1) \cdot ((\mathbf{x}_3 - \mathbf{x}_1) \times (\mathbf{x}_4 - \mathbf{x}_1))$$

Во имплементацијата се зачувува $6V_0$ за да се прескокне делењето со $\frac{1}{6}$ во секоја итерација. Конкретно, **restVolume** во кодот е тројниот продукт на трите вектори на рабовите при конструкција на објектот.

Функцијата на ограничување е

$$C = 6V - 6V_0$$

при што $C > 0$ ако тетраедарот е поголем од почетниот волумен, $C < 0$ ако е компресиран или инвертиран.

4.4.2 Градиенти

Градиентите се векторските производи на спротивните вектори од рабовите на секое теме, односно:

$$\nabla_1 C = (\mathbf{x}_4 - \mathbf{x}_2) \times (\mathbf{x}_3 - \mathbf{x}_2)$$

$$\nabla_2 C = (\mathbf{x}_3 - \mathbf{x}_1) \times (\mathbf{x}_4 - \mathbf{x}_1)$$

$$\nabla_3 C = (\mathbf{x}_4 - \mathbf{x}_1) \times (\mathbf{x}_2 - \mathbf{x}_1)$$

$$\nabla_4 C = (\mathbf{x}_2 - \mathbf{x}_1) \times (\mathbf{x}_3 - \mathbf{x}_1)$$

Геометриски, $\nabla_i C$ покажува надвор од страната која е спроти темето i , со должина еднаква на двапати од плоштината на таа страна. Физички, влечење на темето i нанадвор во таа насока го зголемува волуменот.

4.4.3 Чекор во решавачот

solveVolumeConstraints имплементира:

1. Читање на сите позиции и инверзни маси, излез ако некое теме е неподвижно
2. Пресметка на градиентите од погоре
3. Добивање на моментален $6V$ директно од секој од нив: $6 = \nabla_4 \cdot (\mathbf{x}_4 - \mathbf{x}_1)$ (избегнува повторно пресметување на тројниот продукт)
4. $C = 6V - 6V_0$
5. $\tilde{\alpha} = \frac{\alpha}{\Delta t^2}$
6. $D = w_1 \|\nabla_1\|^2 + w_2 \|\nabla_2\|^2 + w_3 \|\nabla_3\|^2 + w_4 \|\nabla_4\|^2 + \tilde{\alpha}$
7. Излез ако $D < 10^{-8}$ (дегенеративен тетраедар)
8. $\Delta\lambda = \frac{(-C - \tilde{\alpha}\lambda)}{D}$
9. $\Delta x_i = w_i \Delta\lambda \nabla_i$ за $i = 1..4$
10. $\lambda \leftarrow \lambda + \Delta\lambda$

4.4.4 Инверзија и дегенерација

Тетраедар чии темиња се копланарни има нулти волумен и нулта големина на градиентот, при што именителот е само $\tilde{\alpha}$, па ажурирањето дегенерира. Тетраедар што прошол во состојба каде што има негативен волумен (односно е инвертиран) сè уште има добро дефинирани градиенти, а ограничувањето го повлекува назад во валидна состојба. Во практика инверзијата е ретка кога се користат ограничувања на рабови и волумени при нормални временски чекори, кога се случува обично е знак дека временските чекори се предолги или усогласувањето е превисоко.

4.4.5 Сооднос од 0.1 за усогласување

Во конструктурот на **Solver::Solver**, волуменското усогласување е наместено да е $0.1 \times$ рабното усогласување. Ова е хардкодирано, можеби ако се даде како посебен параметар што може да се прилагоди е идно подобрување.....

4.5 Циклусот на XPBD решавачот

Solver::update(dt) е главната функција на целата симулација. Секој повик го зголемува времето за dt (што е времето поминато од една до друга слика) и е поделен на **substeps** број на подчекори. Внатре во секој подчекор се извршува процес од пет фази:

Algorithm 1: Solver::update(dt)

Input: Time step dt , substeps S , solver iterations K

```
1  $h \leftarrow dt/S$ 
2  $e_r \leftarrow 0.3$  // restitution
3  $\mu \leftarrow 0.7$  // friction
4 for  $s = 1$  to  $S$  do
    // 1. Prediction (symplectic Euler)
5    for each particle  $i$  do
6         $\mathbf{v}_i \leftarrow \mathbf{v}_i + h \mathbf{g}$ 
7         $\mathbf{x}_i^{\text{prev}} \leftarrow \mathbf{x}_i$ 
8         $\mathbf{x}_i \leftarrow \mathbf{x}_i + h \mathbf{v}_i$ 
    // 2. Spatial-hash refresh
9     $\mathcal{H}.\text{clear}()$ 
10   for each particle  $i$  do
11        $\mathcal{H}.\text{insert}(i, \mathbf{x}_i)$ 
    // 3. XPBD setup — reset accumulated multipliers
12   for each constraint  $c$  do
13        $\lambda_c \leftarrow 0$ 
    // 4. Constraint solve (Gauss-Seidel)
14   for  $k = 1$  to  $K$  do
15       for each edge constraint  $c$  do
16            $\text{SolveEdge}(c, h)$ 
17       for each volume constraint  $c$  do
18            $\text{SolveVolume}(c, h)$ 
    // 5a. Collision detection + position correction
19   for each particle  $i$  do
20       if  $x_{i,y} < 0$  then  $x_{i,y} \leftarrow 0$ 
21       for each collider  $\mathcal{C}$  do
22            $(d, \mathbf{n}) \leftarrow \mathcal{C}.\text{SignedDistance}(\mathbf{x}_i)$ 
23           if  $d < 0$  then  $\mathbf{x}_i \leftarrow \mathbf{x}_i - d \mathbf{n}$ 
    // 5b. Velocity update (purely positional)
24   for each particle  $i$  do
25        $\mathbf{v}_i \leftarrow (\mathbf{x}_i - \mathbf{x}_i^{\text{prev}})/h$ 
    // 5c. Velocity correction at contacts (restitution + friction)
26   for each particle  $i$  do
27       if  $x_{i,y} \leq 0$  then
28            $v_{i,y} \leftarrow |v_{i,y}| \cdot e_r$ 
29            $v_{i,x} \leftarrow v_{i,x} \cdot \mu$ 
30            $v_{i,z} \leftarrow v_{i,z} \cdot \mu$ 
31       for each collider  $\mathcal{C}$  do
32            $(d, \mathbf{n}) \leftarrow \mathcal{C}.\text{SignedDistance}(\mathbf{x}_i)$ 
33           if  $d < d_{\text{contact}}$  then
34                $\mathbf{v}_n \leftarrow (\mathbf{v}_i \cdot \mathbf{n}) \mathbf{n}$ 
35               if  $\mathbf{v}_i \cdot \mathbf{n} < 0$  then
36                    $\mathbf{v}_t \leftarrow \mathbf{v}_i - \mathbf{v}_n$ 
37                    $\mathbf{v}_i \leftarrow -e_r \mathbf{v}_n + \mu \mathbf{v}_t$ 
```

5. Техники за детекција на колизии

Симулатор на меки тела во реално време е, суштински, циклус кој се движи низ честичките и одговара на прашањето „дали оваа честичка е во контакт со било што?“ за илјадници честички во секој потчекор. Наивниот одговор е да се тестира контакт на честичката со секој триаголник на секое тело со кое е можно да има контакт, плус сите други честички од телото за да се провери само-судир. Ова има комплексност од $O(N \cdot T)$ во секој потчекор, каде N е бројот на честички и T е вкупниот број на триаголници на статичните објекти. Ако N е во опсегот на неколку илјади а T во опсегот на десетици илјади, многу брзо го надминуваме буџетот за пресметка во една слика за да постигнеме 60 слики во секунда.

Стандардното решение е да се подели детекцијата на судири во широка фаза и тесна фаза. Во широката фаза се користат едноставни граници на објектите (како axis-aligned bounding boxes - AABBs) за да се одбие поголемиот дел од паровите што немаат шанса да бидат во контакт. Потоа во тесната фаза се користат попрецизни и поскапи методи за да се тестираат само можните кандидати за колизии.

5.1 Забрзување во широка фаза (broad-phase acceleration)

Постојат четири главни структури за забрзување кои се користат во симулациите во реално време.

Sweep-and-prune[16][17]. Се одржуваат три подредени листи со крајни точки на AABB, по една за секоја оска. Две AABB се преклопуваат ако и само ако се преклопуваат на сите три оски. Вметнувањата и бришењата се локални; кај умерено динамични сцени, подредените листи ретко се менуваат (претпоставка за временска кохерентност). SAP се истакнува кај цврсти тела кои се движат умерено помеѓу сликите; плаќа трошок за прераспоредување кога геометријата агресивно се деформира.

Хиерархиски структури на ограничувачки волумени (BVH). Бинарно дрво од вградени AABB (или OBB / сфери / k-DOP) каде што секој лист опфаќа

по еден примитив. Пребарувањето за најблизок објект до некоја е спуштање со DFS кое ги отстранува сите поддрва чија AABB е веќе подалеку од тековната најдобра вредност. Трошокот за градење е $O(T \log T)$, а просечниот трошок за пребарување е $O(\log T)$. Овој пристап е природниот избор за статична геометрија која се пребарува многупати.

Униформно просторно хаширање [5]. 3-Д просторот се мапира на регуларна мрежа од ќелии со страна s , се хешира целобројната координатата на ќелијата во табела со фиксна големина и секој примитив се вметнува во своја кофа на ќелијата. Пребарувањата по честичка ги испитуваат 27-те околни ќелии (3^3 за да се обработат граничните ефекти). Вметнувањето е $O(1)$; барањето е $O(k)$, каде k е бројот на зафатени ќелии. Нема дрво, нема сортирање, нема прилагодување — добро прилагодени за множества што брзо се деформираат каде што секој примитив се движи на секој чекор.

Опуштени ок-дрва и k-d дрва (octrees and k-d trees). Хиерархиски поделби на просторот; корисни кога сцената има многу нерамномерна густина. Изградбата и пречекорувањето се посложени и од BVH и од хеш; ретко се привлечни за работните оптоварувања во оваа теза.

Во овој труд има две различни работни оптоварувања кои користат две различни структури:

Статичен колајдер од триаголна мрежа. Се гради еднаш, се пребарува многупати. BVH е стандардниот избор бидејќи се плаќа трошокот за градење однапред, а се амортизира преку милиони барања.

Површината на деформирано меко тело. Нејзините површински честички се движат на секој потчекор; повторното градење на BVH во секој чекор би било излишно. Униформниот просторен хеш се реконструира во $O(N)$ по чекор и му овозможува на статичниот колајдер да ги најде карактеристиките на површината во близина на секоја од неговите сопствени темиња и рабови.

Овој образец на различни структури за различни оптоварувања, избрани одделно врз основа на тоа колку често се менува геометријата е стандард во енципите за реално време. Bullet, на пример, користи btDbvtBroadphase (динамички BVH) за цврсти тела и посебен мрежа на ќелии за проверки на меки тела.

5.1.1 BVH за статични мрежи

Во `MeshCollider::buildBVH` се гради класичен BVH:

1. Иницијализаци на коренски јазол чиј AABB го содржи секој триаголник во влезната мрежа
2. Ако има само еден триаголник,значи го овој јазол како лист, зачувај го индексот на триаголникот и излези
3. Ако не, пресметај AABB на центроидите на триаголниците во влезот. Избери ја најдолгата оска на AABB како оска на поделба

4. Искористи `std::nth_element` за да се подели низата на индекси на триаголници така што првата половина има помали координати на центроидот во однос на оската на поделба од втората половина. `nth_element` е $O(n)$ во просек, што ни дава $O(n \log n)$ комплексност за градење на структурата
5. Направи рекурзија во двете половини за да се изградат левото и десното поддрво, зачувај ги индексите во јазолот

Барање (queryBVH):

1. Ако квадрираната далечина од AABB на тековниот јазол до точката е веќе полоша од `bestSqDist`, исечи ја гранката
2. Ако јазолот е лист, изврши `closestPointOnTriangle` (§5.3.1) и ажурирајте ги `bestSqDist`, `bestTriIdx`, `bestFeature`, `bestFeatureData`, `bestClosest` ако овој триаголник е подобар од тековниот најдобар.
3. Во спротивно, пресметај `sqDistanceTo(point)` за двете деца и прво посети го поблиското. Посетата на поблиското прво обично дава помала најдобра квадратно растојание, што поефикасно го отстранува пооддалеченото поддрво.

`AABB::sqDistanceTo` е стандардната формула "clamp-and-norm": `clamp(point, min, max)` ја проектира точката на затворената кутија; квадратот на должината на остатокот е квадратот на растојанието. Побрзо од пресметување на растојанието и негово квадрирање.

5.1.2 Просторен хаш за површината на деформираното меко тело

`SpatialHash` (во `include/physics/spatial_hash.hpp`) е мал и самостоен:

Координата на ќелијата. `cellCoord(pos) = floor(pos / cellSize)` as `glm::ivec3`. `glm::floor` заокружува кон негативната бесконечност, така што негативните координати се мапираат правилно без погрешка од еден помалку на нула.

Хеш-функција. $(x \cdot 92837111)(y \cdot 689287499)(z \cdot 283923481)$, а потоа модуло `tableSize`. Трите множители се големи прости броеви од `Teschner 2003`.

Безбедност на знакот при модуло. $(h \% tableSize + tableSize) \% tableSize$ гарантира ненегативен индекс на кофа, дури и кога `h` е негативен — што е важно бидејќи операторот `%` во C++ со цели броеви може да врати негативна вредност.

Колизија (во смисла на хаширање). Различни ќелии може да бидат хеширани во истиот кош; кошевите се вектори `<int>` од секоја честичка што завршила таму, без оглед на ќелијата. Пребарувањата ги проверуваат честичките на ќелиите и применуваат тестови за растојание по парови во тесната фаза, така што судирите при хаширање чинат најмногу дополнителна работа во тесната фаза, но никогаш не ја нарушуваат исправноста.

Хашот се реконструира на почетокот на секој потчекор во однос на *предвидените* позиции (позициите по чекорот на интеграција, но пред решавањето на ограничувањата). Ова е правилната конфигурација за пребарување во тесната фаза:

решавачот на ограничувања потоа ги коригира позициите кон резултатите од тие пребарувања.

Хешот се користи од страна на `Solver::buildColliderCollisionCandidates`. Во секоја слика, тој ги вметнува честичките од површината на мекото тело, а потоа ги итерира темињата и рабовите на статичкиот меш-колајдер. За секој теме на колајдерот, тој го пребарува хешот на позицијата на тоа теме за да најде блиски површински честички, ги проширува тие честички во нивните инцидентни површински триаголници (преку однапред пресметаното врв-триаголник поврзаност) и ги задржува оние што се во рамките на контактната маргина како врв-триаголник (VT) кандидатски парови. За секој раб на колајдерот се прави истото со рабовите на површината (поврзаност теме-раб), при што се добиваат кандидати-парови раб-раб (EE). Бидејќи колајдерот е статичен во рамките на една слика, оваа широка фаза се извршува еднаш (во првиот потчекор) и добиените кандидатски парови повторно се користат во преостанатите потчекори; тестот во тесната фаза на еден пар ги повторно користи примитивите ”најблиска точка на триаголникот” и ”најблиска точка помеѓу сегментите” од §5.3.1, а одговорот се применува во Поглавје 6.

Големината на ќелиите е единствениот параметар и го контролира компромисот: ако ќелијата е премала тогаш ќелијата е празна поголемиот дел од времето и пребарувањата покриваат премал регион, од друга страна ако ќелијата е преголема многу честички ќе се содржат во една ќелија, па пребарувањата на широк опсег ќе тежнеат кон груба сила. Во имплементацијата се користи `cellSize` = просечна должина во мирување на рабовите. Ова ни дава отприлика една честичка по ќелија, што е блиску до оптимално.

Табела 5.1: Споредба на структури за забрзување на широка фаза

Структура	Градење	Пребарување (по честичка)	Цена за реизградба	Се совпаѓа со геометрија што се деформира
Груба сила	нема	$\mathcal{O}(T)$ или $\mathcal{O}(N)$	нема	да, но прескапо за користење
SAP	$\mathcal{O}(T \log T)$ еднаш	$\mathcal{O}(\log T)$ амортизирано	$\mathcal{O}(T)$ по чекор	лошо
BVH	$\mathcal{O}(T \log T)$ еднаш	$\mathcal{O}(\log T)$	$\mathcal{O}(T)$ за нагудување, $\mathcal{O}(T \log T)$ реизградба	со нагудување, реизградба за голема деформација
Просторен хеш	$\mathcal{O}(N)$ по чекор	$\mathcal{O}(1)$ амортизирано	нема	одлично

5.2 Тесна фаза: аналитички примитиви

5.2.1 SDF апстракција

Во симулаторот, главна комуникација помеѓу колајдерите и решавачот е функцијата `Collider::signedDistance` која што враќа растојание помеѓу честичка и колајдерот како и нормалниот вектор во таа точка. Растојанието може да се пресмета

на повеќе начини: аналитички или со изминување на BVH, но на решавачот му е битно само растојанието и нормалата.

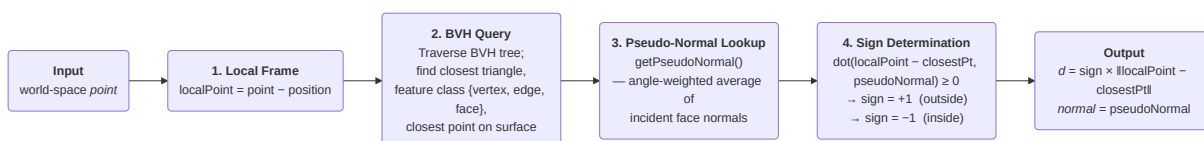
5.2.2 SDF сфера

Еден од двата типови на пресметување растојание е аналитички со користење на SDF. Ова е прикажано со **SphereCollider** во имплементацијата. Предноста на SDF е што нема потреба од барање на најблиска точка и претходни пресметки за структури за забрзување, тие имаат дефинирани функции кои го враќаат растојанието од произволна точка до површината на телото. Постојат цели библиотеки на аналитички SDFs од кои сферата е наједноставна, други се: рамнина, кутија, цилиндар, капсула итн. Тие дополнително можат да се комбинираат со унија, пресек и разлика.

Недостатокот за SDFs е што не постои формула за растојание за произволна триаголнеста мрежа. Во наредниот дел се објаснува стандардниот начин на надминување на проблемот: изминување на геометријата за да се најде најблизок триаголник и комбинирање на резултатот од пребарување на најблиска точка на триаголникот со нормала за да се добие растојание со знак.

5.2.3 Тесна фаза: колизија на триаголник и мрежа

Процесот на наоѓање најблиска точка за произволна мрежа е следниот:



Функцијата **MeshCollider::signedDistance** го имплементира овој процес. Чекор 2 е објаснет во 5.1.1, а останатите делови ќе бидат опишани во остатокот од подглавата.

5.2.4 Најблиска точка на триаголник (Воронои метод)

За даден триаголник со темиња $\vec{a}$, $\vec{b}$, $\vec{c}$ и точка $\vec{p}$, најблиската точка на триаголникот лежи во точно еден од седум региони во стил на Воронои:

- Три региони на темиња: еден по теме, при што $\vec{p}$ е најблиску до темето
- Три региони на рабови: еден по раб, $\vec{p}$ е најблиску до раб
- Еден регион на страната: $\vec{p}$ е најблиску до проекцијата на рамнината на триаголникот (која што лежи во триаголникот)

Ериксон [18] дава каскада на скаларни производи што класифицира во кој регион спаѓа $\vec{p}$ и ја враќа најблиската точка во $O(1)$. Во имплементацијата се користи овој метод.

5.2.5 Проблем на дисконтинуитет на знак

Природниот начин да се одреди знакот е да се направи скаларен производ на точката и нормалата страната на најблискиот триаголник. Ако резултатот е позитивен, точката е на надворешната страна; ако е негативен, на внатрешната страна. За точка во Воронои регионот на лицето на триаголник, ова е точно: нормалата на лицето е вистинската надворешна нормала таму.

Меѓутоа, за точка во регион на работ или во регион на теме, нормалата на лицето е погрешна. Два соседни триаголници кои се спојуваат на остар раб имаат различни нормали на лицето; една точка која се наоѓа во близина на работ може да се класифицира како внатрешна во однос на едниот и како надворешна во однос на другиот. Уште полошо, BVH враќа еден од двата соседни триаголници како најблизок, но мали движења на точката можат да го променат триаголникот што се враќа, менувајќи го знакот додека точката го поминува работ, види фигура 5.1

Феноменот е лесен за демонстрирање. Ако се следи точка на по лак што го допира работ на единечна коцка. Со знакот на нормалата на лицето, далечината има прекин (дисконтинуитет во изводот) при преминот на работ, а кај остри диедрали самиот знак може да се смени иако барањето е недвосмислено надвор од коцката.

За одговорот при судир (Поглавје 6), последицата е видлива: честичка близу до работ се турка во погрешна насока, погрешниот знак на корекција ја испраќа низ површината и симулацијата се расипува. Ова беше оригиналната мотивација за псевдо-нормалната конструкција.

5.2.6 Псевдо-нормали

Беренцен и Анес [19] покажуваат дека конзистентно поле на нормали на површината, односно една нормала на страна, раб и теме, може да се дефинира така што тестот со скаларен производ го дава точниот резултат за *секоја* точка на проверка. Нормалите на рабовите и темињата мора внимателно да се конструираат од тежински просеци на околните нормали на страните.

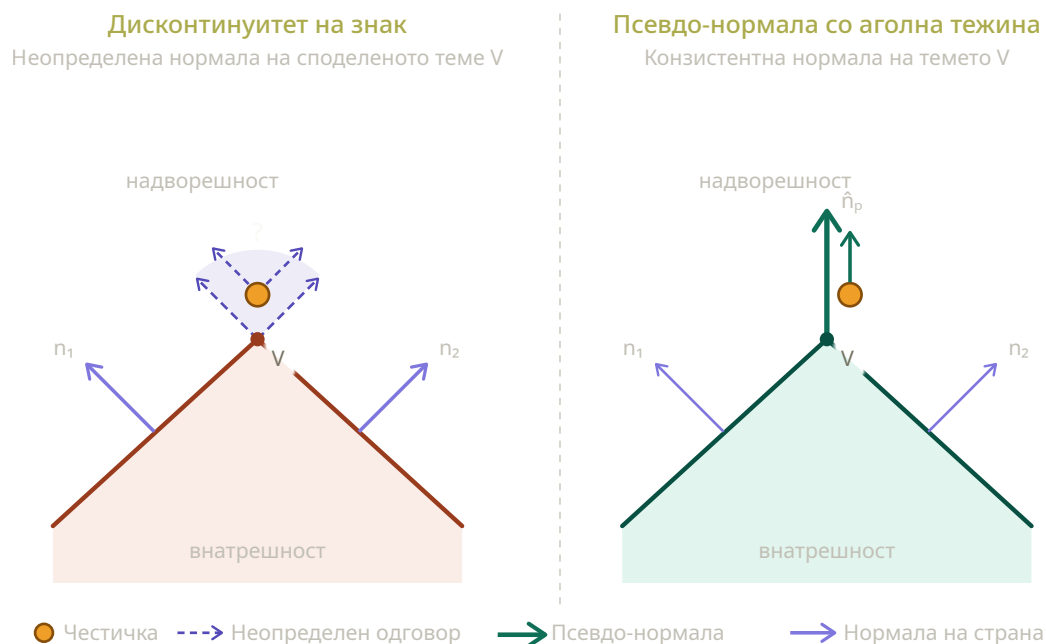
Псевдо-нормала на теме, за секое теме v се сумираат нормалите на секој триаголник чие теме е v , со тежина еднаква на внатрешниот агол на тој триаголник кај темето v :

$$\vec{N}_v = \frac{(\sum_i \theta_i \vec{n}_i)}{\|\sum_i \theta_i \vec{n}_i\|}$$

каде сумата е врз инцидентните триаголници, $\vec{n}_i$ е нормалата на триаголникот, и θ_i е аголот на триаголникот i кај v . Тежината по агол ја прави псевдо-нормалата инваријантна на промена на инцидентните триаголници.

Псевдо-нормала на раб. За секој раб се зема просек од нормалите на двата соседни триаголници:

$$\vec{N}_e = \frac{\vec{n}_l + \vec{n}_r}{\|\vec{n}_l + \vec{n}_r\|}$$



Слика 5.1: Дисконтинуитет на знак наспроти псевдо-нормала со аголна тежина кај споделено теме на два триаголници

Пребарување, `getPseudoNormal` на влез го добива типот на карактеристика (раб, теме, триаголник) и ја враќа соодветната псевдо-нормала.

5.2.7 Резултат

За секоја честичка, повикот до **`MeshCollider:signedDistance`** враќа растојание со знак на честичката до колајдерот (позитивно ако точката е надвор од објектот, негативно ако е внатре) и единечна нормала (со насока нанадвор). Ова подоцна се користи од решавачот за пресметување на одговор на евентуалната колизија (Поглавје 6). Цената при пребарување е доминирана од изминување на BVH дрвото: $O(\log T)$ во просек, со мала константна комплексност од пресметката за најблиска точка на секој лист на дрвото и тестот за растојание за интерните јазли на AABB.

Ограничување е што симулаторот претпоставува дека мрежата е затворена, односно нема да работи за мрежи/објекти кои се отворени, на пример чашка, бидејќи нема добро дефиниран внатрешен и надворешен дел.

6. Техники за одговор на колизија

Детекцијата на колизија од претходната глава дава 3 информации на фазата за одговор:

- **Длабочина на пенетрација** d - колку длабоко навлегла честичката во телото
- **Нормала на точката на контакт** $\vec{n}$ - нормализирана насока
- **Точка на контакт** (најчесто иста со моменталната позиција на честичката)

Во фазата на одговор треба:

- Да се одделат телата, односно да доведе d на 0 со поместување на едно од телата
- Да се промени нормалната брзина, односно да сведе на нула (нееластично), да се сврти обратно (еластично), или некаде помеѓу со коефициентот на еластичност
- Да се промени тангентната брзина, односно да се намали со триење со што се намалува кинетичката енергија

Има четири парадигми за решавање на проблемот кои се разликуваат во одлуката кога и како се прават промените

6.0.1 Методи со пенали

Најстар и најинтуитивен пристап[10], се третира продирањето како компресирана пружина и се применува сила на реставрирање $F = -k \cdot d \cdot \vec{n}$, со термин на намалување $F_d = -c \cdot (\vec{v} \cdot \vec{n})\vec{n}$ за да се намали нормалната брзина. Силата влегува во равенките за движење како сила на телото кај контактот, интегрирана со временскиот чекор што го користи системот.

Тие се лесни за имплементација, може да се користат во било кој тип на динамика (експлицитен Ојлер, RK4, полуимплицитен итн.) и се диференцијабилни, што е важно за апликации базирани на градиенти. Но имаат проблем со цврстината, за да се намали продирањето, k мора да биде големо, но големо k ја дестабилизира

интеграцијата, па на наидува на проблем на видлива мекост за мало k и експлодирање на симулацијата за големо k . Друг проблем е што може да се внесе енергија во системот и што има осцилација околу точката на контакт поради користењето на пружини.

Овие методи се важни историски но не се користат во денешните апликации.

6.0.2 Импулсни методи

Наместо да се примени сила врз конечен временски интервал, се применува момен-тен импулс J на контактната точка што директно ја менува брзината. Големината на импулсот се пресметува од :

$$(\vec{v}_i' - \vec{v}_j') \cdot \vec{n} = -e \cdot (\vec{v}_i - \vec{v}_j) \cdot \vec{n}$$

каде $\vec{v}_i' = \vec{v}_i + J \cdot \frac{\vec{n}}{m_i}$ и e е коефициентот на реституција.

Предностите им се што немаат параметар за цврстина и реституцијата е едно-ставна физичка мерка од $[0,1]$ и за еден контакт има само една пресметка, без инте-грирање. Тие се стандард во енџини за цврсти тела. Слабостите им се што кога има повеќе истовремени контакти има лошо скалирање на цената за пресметка. Тие се одлични за ретки контактни точки кај цврсти тела, но неа за густе и повеќебројни контакти кај меките тела

6.0.3 Контакт базиран на ограничувања

Се третира ској контакт како ограничување со нееднаквост

7. Имплементација и рендерирање

7.1 Технологии

Симулаторот е напишан во C++, стандардниот избор во пишување на физички симулатори заради брзината и развиениот екосистем на библиотеки за компјутерска графика. Библиотеките што се користат се:

- **OpenGL 4.3** за рендерирање, избран заради едноставноста и можноста да се додадат compute shaders во иднина
- **GLFW** за креирање прозорец, контекст на OpenGL и читање на влезни настани. Работи на повеќе платформи и е модерен
- **GLAD** за вчитување на OpenGL функциите
- **GLM** за линеарна алгебра (операции со матрици и вектори), лесна интеграција со GLSL векторските типови: glm::vec3, glm::mat4, glm::cross итн.
- **Assimp** за вчитување на мрежите. Конфигуриран само за вчитување на OBJ, FBX, GLTF датотеки за намалување на големина
- **TetGen** за тетраедрализацијата на мрежите
- **ImGui** за приказ на едноставен кориснички интерфејс

Овие технологии беа избрани бидејќи овозможуваат лесно извршување на апликацијата на различни платформи со минимална конфигурација.

7.2 Структура на модули

Апликацијата е поделена на 4 модули кои имаат своја улога.

Физика, јадрото на симулаторот. Нема знаење за OpenGL и приказот на екран. Добива објекти и почетна состојба на влез (меки тела и колајдери), изложува функција за ажурирање на состојбата на објектите во одреден временски интервал и изложува функција за добивање на моменталната положба на сите објекти за да се прикажат од рендерерот. Дополнително се изложуваат методи кои се повикуваат од корисничкиот интерфејс.

Рендерирање, содржи структури за додавање објекти во OpenGL сцената

Библиографија

- [1] Xavier Provot. Deformation constraints in a mass-spring model to describe rigid cloth behavior. 1995. URL: <https://api.semanticscholar.org/CorpusID:12704543>.
- [2] David Baraff and Andrew Witkin. Large steps in cloth simulation. In *Proceedings of the 25th Annual Conference on Computer Graphics and Interactive Techniques*, SIGGRAPH '98, page 43–54, New York, NY, USA, 1998. Association for Computing Machinery. doi:10.1145/280814.280821.
- [3] Matthias Müller, Bruno Heidelberger, Marcus Hennix, and John Ratcliff. Position based dynamics. *J. Vis. Comun. Image Represent.*, 18(2):109–118, April 2007. doi:10.1016/j.jvcir.2007.01.005.
- [4] Miles Macklin, Matthias Müller, and Nuttapong Chentanez. Xpbd: position-based simulation of compliant constrained dynamics. In *Proceedings of the 9th International Conference on Motion in Games*, pages 49–54, 2016.
- [5] Matthias Teschner, Bruno Heidelberger, Matthias Müller, Danat Pomerantes, and Markus H Gross. Optimized spatial hashing for collision detection of deformable objects. In *Vmv*, volume 3, pages 47–54, 2003.
- [6] Matthias Müller and Markus Gross. Interactive virtual materials. In *Proceedings of Graphics Interface 2004*, GI '04, page 239–246, Waterloo, CAN, 2004. Canadian Human-Computer Communications Society.
- [7] Thomas Jakobsen. Advanced character physics. In *Game developers conference*, volume 3, pages 383–401. IO Interactive, Copenhagen Denmark, 2001.
- [8] Matthias Müller, Bruno Heidelberger, Matthias Teschner, and Markus Gross. Meshless deformations based on shape matching. *ACM Trans. Graph.*, 24(3):471–478, July 2005. doi:10.1145/1073204.1073216.
- [9] Miles Macklin and Matthias Müller. Position based fluids. *ACM Transactions on Graphics (TOG)*, 32(4):1–12, 2013.
- [10] James K Hahn. Realistic animation of rigid bodies. *ACM Siggraph computer graphics*, 22(4):299–308, 1988.

- [11] Brian Mirtich. Impulse-based dynamic simulation of rigid body systems. 1996. URL: <https://api.semanticscholar.org/CorpusID:28130224>.
- [12] Robert Bridson, Ronald Fedkiw, and John Anderson. Robust treatment of collisions, contact and friction for cloth animation. In *Proceedings of the 29th annual conference on Computer graphics and interactive techniques*, pages 594–603, 2002.
- [13] Matthias Müller, Miles Macklin, Nuttapong Chentanez, Stefan Jeschke, and Tae-Yong Kim. Detailed rigid body simulation with extended position based dynamics. In *Computer graphics forum*, volume 39, pages 101–112. Wiley Online Library, 2020.
- [14] Thomas Larsson and Tomas Akenine-Möller. Collision detection for continuously deforming bodies. In *Eurographics (short presentations)*, 2001.
- [15] Hang Si. Tetgen, a delaunay-based quality tetrahedral mesh generator. *ACM Trans. Math. Softw.*, 41(2), February 2015. doi:10.1145/2629697.
- [16] David Baraff and Andrew Witkin. Dynamic simulation of non-penetrating flexible bodies. In *Proceedings of the 19th Annual Conference on Computer Graphics and Interactive Techniques*, SIGGRAPH '92, page 303–308, New York, NY, USA, 1992. Association for Computing Machinery. doi:10.1145/133994.134084.
- [17] Jonathan D. Cohen, Ming C. Lin, Dinesh Manocha, and Madhav Ponamgi. I-collide: an interactive and exact collision detection system for large-scale environments. In *Proceedings of the 1995 Symposium on Interactive 3D Graphics*, I3D '95, page 189–ff., New York, NY, USA, 1995. Association for Computing Machinery. doi:10.1145/199404.199437.
- [18] Christer Ericson. *Real-time collision detection*. Crc Press, 2004.
- [19] Jakob Andreas Bærentzen and Henrik Aanaes. Signed distance computation using the angle weighted pseudonormal. *IEEE Transactions on Visualization and Computer Graphics*, 11(3):243–253, 2005.